

LAB6 Time-Frequency Transforms

Task1

```
%% task1
function DFT = isIDFT(signal)
    N = length(signal);    % Signal length
    DFT = zeros(1, N);
    i = sqrt(-1);
    for k = 0:N-1
        DFT(k+1) = sum(signal .* exp(-2*pi*i*k*(0:N-1)/N));
    end
end
```

```
%% Chirp Signal
```

```
clc
```

```
clear all
```

```
close all
```

```
Fs = 1000; % Sampling frequency
```

```
dt = 1/Fs; % Time step
```

```
T = 1; % Total time
```

```
t = 0:dt:T; % Time vector
```

```
f_start = 20; % Start frequency
```

```
f_stop = 220; % Stop frequency
```

```
s = chirp(t, f_start, T, f_stop); % chirp signal
```

```
figure(1); % Plot the chirp signal
```

```
plot(t, s);
```

```

xlabel('Time (s)');
ylabel('Amplitude');
title('Chirp Signal');

% Compute FFT using MATLAB's built-in function
N = length(s); % Number of samples
df = Fs/N; % Frequency step
f_vec = (0:(N-1))*df; % Frequency vector

S = fft(s); % Calculate fft for signal s
myfft_amp = abs(S);
S_phase = atan2(imag(S),real(S)); % phase
figure(2);
subplot(2,1,1);
plot(f_vec, myfft_amp);
xlabel('Frequency [Hz]'); ylabel('Amplitude');
title('Amplitude Spectrum (MATLAB fft)');
subplot(2,1,2);
plot(f_vec, S_phase);
xlabel('Frequency [Hz]'); ylabel('Phase');
title('Phase Spectrum (MATLAB fft)');

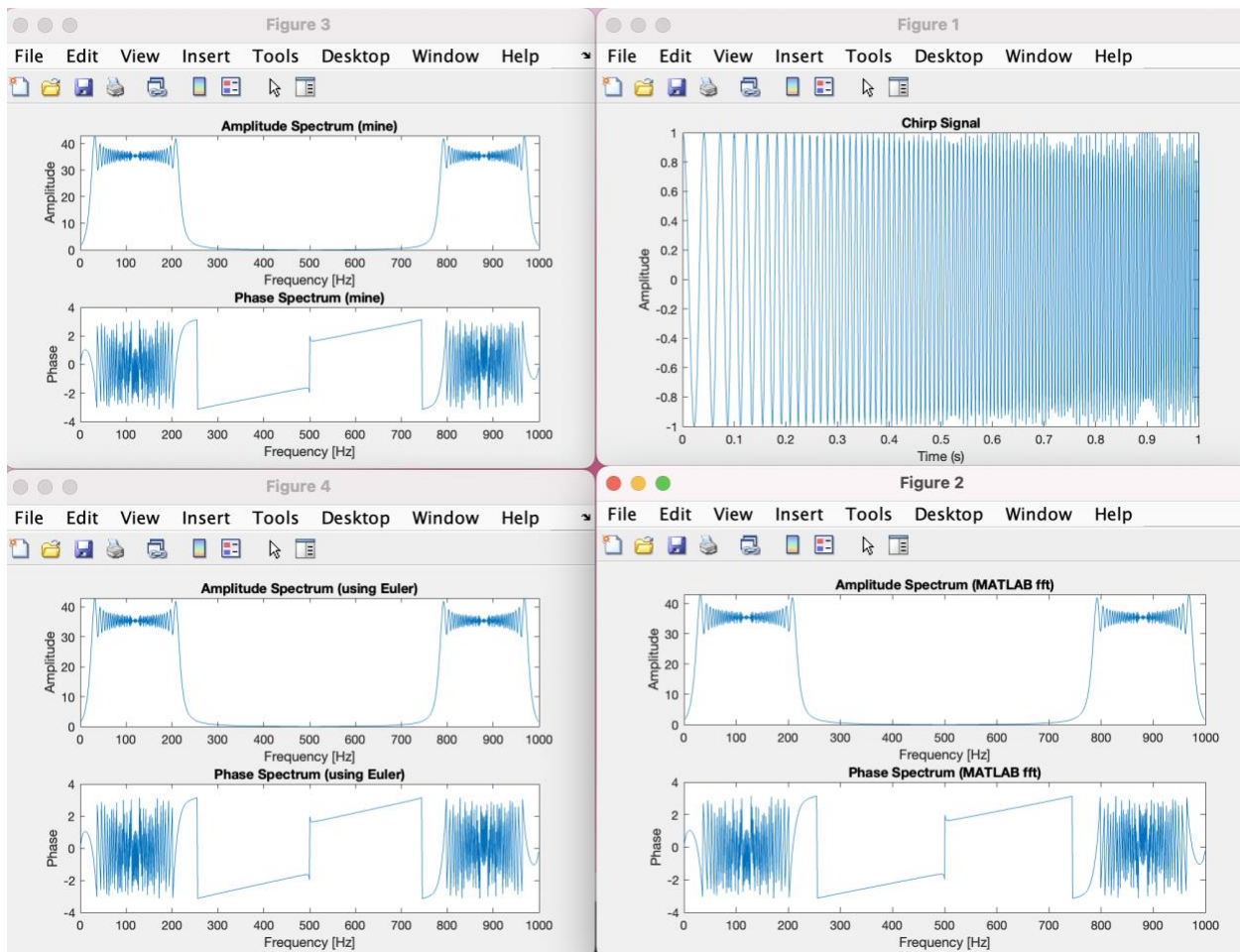
% using my DFT implementation
DFT_result = isilDFT(s);
DFT_amp = abs(DFT_result);
DFT_phase = angle(DFT_result);
figure(3);
subplot(2,1,1);
plot(f_vec, DFT_amp);
xlabel('Frequency [Hz]');
ylabel('Amplitude');
title('Amplitude Spectrum (mine)');
subplot(2,1,2);

```

```
plot(f_vec, DFT_phase);  
xlabel('Frequency [Hz]');  
ylabel('Phase');  
title('Phase Spectrum (mine)');
```

```
% Euler's
```

```
Euler_result = isIDFTEuler(s);  
Euler_amp = abs(Euler_result);  
Euler_phase = angle(Euler_result);  
figure(4);  
subplot(2,1,1);  
plot(f_vec, Euler_amp);  
xlabel('Frequency [Hz]');  
ylabel('Amplitude');  
title('Amplitude Spectrum (using Euler)');  
subplot(2,1,2);  
plot(f_vec, Euler_phase);  
xlabel('Frequency [Hz]');  
ylabel('Phase');  
title('Phase Spectrum (using Euler)');
```



Task2

%% Time and frequency vectors

clc

clear all

close all

Fs = 1000; % Sampling frequency

f1 = 50;

f2 = 51;

T1 = 1;

T2 = 10;

t1 = (0:Fs*T1-1)/Fs;

```

t2 = (0:Fs*T2-1)/Fs;
s1 = sin(2*pi*f1*t1) + sin(2*pi*f2*t1); % Signal 1
s2 = sin(2*pi*f1*t2) + sin(2*pi*f2*t2); % Signal 2

N1 = length(s1);
df1 = Fs/N1;
f1_vec = (0:(N1-1))*df1;
S1 = fft(s1);
S1_amp = abs(S1);

N2 = length(s2);
df2 = Fs/N2;
f2_vec = (0:(N2-1))*df2;
S2 = fft(s2);
S2_amp = abs(S2);

figure(1);
subplot(2, 1, 1); % Plot Signal 1
plot(f1_vec, S1_amp, 'b', 'LineWidth', 1.5);
title('Amplitude-Frequency Spectrum (Signal 1, T = 1 sec)');
xlabel('Frequency (Hz)');
ylabel('Amplitude');
xlim([0, Fs/2]);

subplot(2, 1, 2); % Plot Signal 2
plot(f2_vec, S2_amp, 'r', 'LineWidth', 1.5);
title('Amplitude-Frequency Spectrum (Signal 2, T = 10 sec)');
xlabel('Frequency (Hz)');
ylabel('Amplitude');
xlim([0, Fs/2]);

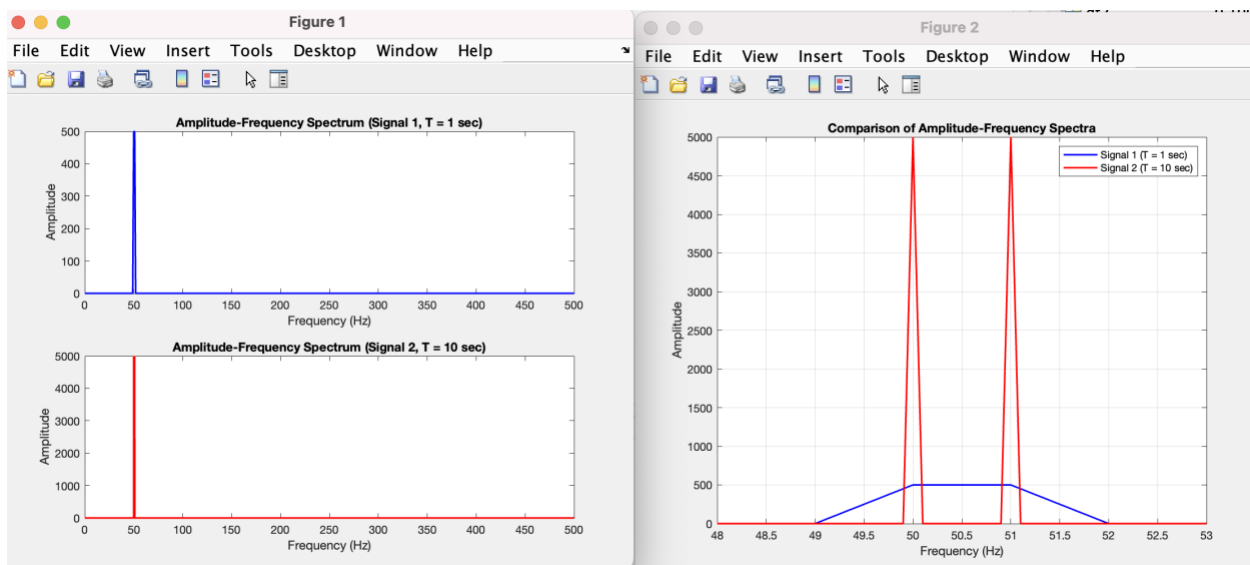
figure(2); % plot: Signals 1 vs 2
plot(f1_vec, S1_amp, 'b', 'LineWidth', 1.5);

```

```

hold on;
plot(f2_vec, S2_amp, 'r', 'LineWidth', 1.5);
hold off;
title('Comparison of Amplitude-Frequency Spectra');
xlabel('Frequency (Hz)');
ylabel('Amplitude');
legend('Signal 1 (T = 1 sec)', 'Signal 2 (T = 10 sec)');
xlim([48, 53]);
grid on;

```



Signal 2 t10 > amplitude than signal1 t1

Task3

```
clc
```

```
clear all
```

```
close all
```

```
Fs = 1000; % sampling frequency
```

```
T = 1; % signal duration
```

```
t = 0:1/Fs:T-1/Fs; % Time vector
```

```
signal = chirp(t, 20, T, 220); % chirp signal
```

```
figure(1);  
plot(t,signal)  
xlabel('Time (s)'), ylabel('Amplitude');
```

```
% Spectrogram 1 time window length 0.25s, 0% overlap  
% MATLAB's built-in spectrogram function  
WIN1 = round(windowLength1 * Fs);  
PRZES1 = round(WIN1 * (1 - overlap1));  
[S1,F1,To1] = spectrogram(signal, WIN1, WIN1 - PRZES1, WIN1, Fs);  
figure(3);  
imagesc(To1, F1, abs(S1));  
ylim([0 500]);  
axis xy;  
title('MATLAB Spectrogram (0.25s window, 0% overlap)');  
xlabel('Time (s)');  
ylabel('Frequency (Hz)');  
colorbar;
```

```
% Spectrogram 2 time window length 0.1s, 50% overlap  
% MATLAB's built-in spectrogram function  
WIN2 = round(windowLength2 * Fs);  
PRZES2 = round(WIN2 * (1 - overlap2));  
[S2,F2,To2] = spectrogram(signal, WIN2, WIN2 - PRZES2, WIN2, Fs);  
figure(5);  
imagesc(To2, F2, abs(S2));  
ylim([0 500]);  
axis xy;  
title('MATLAB Spectrogram (0.1s window, 50% overlap)');  
xlabel('Time (s)');  
ylabel('Frequency (Hz)');
```

colorbar;

